BERGHS · ACE2627 · PROGRAMMING & METHODS

HOW MY PORTFOLIO SAVES DATA

Posts, comments and logins in a Neon Postgres database

AXEL AF UGGLAS

INTRODUCTION

THE SITE

A portfolio for my AI video work and my CV, shown as a 3D cube you move around in.

Each video work is a blog post. I add posts in a studio behind a login; visitors leave comments.

STACK

Node.js + Express server

HTML, CSS, JS pages

Postgres on Neon database

Cloudinary videos, images

Claude API AI summaries

NOAA Earth's magnetic field

INTRODUCTION

WHY A DATABASE?

Without one

A new post lives only in the browser's memory. Refresh the page and it's gone. Nobody else ever sees it.

With one

Posts and comments are saved on a server in the cloud. They survive refreshes and restarts, and every visitor sees the same data.

Today you'll learn: where the data lives · how it gets there · how it's kept safe

HOW IT WORKS

WHERE IT LIVES: NEON

Neon is PostgreSQL in the cloud, on the free tier.

PostgreSQL: a database that keeps data in tables of rows and columns.

SQL: the language you use to ask it to read or save data.

1 db/schema.sql describes the three tables.

2 npm run db:setup creates them. “IF NOT EXISTS” makes it safe to run again.

3 db.js opens one shared connection pool to Neon’s pooled address, so requests reuse connections.

4 The address, DATABASE_URL, lives in .env.

HOW IT WORKS

THREE TABLES

USERS

id
username
password_hash

One row: me. A bcrypt hash, never the password.

POSTS

id · slug · title · year
position · video_url
tags (TEXT[])
process (JSONB)
reference_images (JSONB)
summary
created_at · updated_at

One row per video work.

COMMENTS

id
post_id → posts.id
first_name · last_name
email · body
created_at

Delete a post and its comments go too (ON DELETE CASCADE).

Primary key (id): each row's unique number. **Foreign key** (post_id): points at a row in another table.

HOW IT WORKS

THE JOURNEY OF A COMMENT

1 BROWSER
A visitor writes a comment and presses send.

→

2 FETCH()
The page sends the fields to the server as JSON.

→

3 EXPRESS
The route checks and trims every field.

→

4 SQL
An INSERT with $1…$5 placeholders.

→

5 NEON
Saves the row and sends it back; the page shows it.

Route: POST /api/posts/:id/comments

Reading is the same trip in reverse: GET and SELECT.

HOW IT WORKS

WHO MAY DO WHAT

Method	Route	Who	What it does
GET	/api/posts	Everyone	Lists all works; builds the cube
POST	/api/posts	Signed in	Adds a work
PUT	/api/posts/:id	Signed in	Edits a work
DELETE	/api/posts/:id	Signed in	Deletes a work and its comments
GET	/api/posts/:id/comments	Everyone	Reads comments
POST	/api/posts/:id/comments	Everyone	Writes a comment

Every write route runs requireLogin first: no session, no change.

MY IMPLEMENTATION

SAVING A COMMENT: THE CODE

```
// comments.js
const { rows } = await db.query(
  `INSERT INTO comments
     (post_id, first_name, last_name, email, body)
   VALUES ($1, $2, $3, $4, $5)
   RETURNING ${PUBLIC_COLUMNS}`,
  [req.params.id, firstName, lastName, email, body]
);
```

$1...$5 are placeholders. The values travel separately from the SQL, so a comment like "; DROP TABLE posts" is saved as text, never run.

RETURNING sends back only public columns: id, first name, comment and date. The email stays in the database.

MY IMPLEMENTATION

KEEPING IT SAFE

Parameterised queries

Visitor text never becomes part of the SQL itself. That blocks SQL injection.

The server checks again

The form checks the fields, but anyone can call the API directly. The server trims, limits length and validates the email.

Emails stay private

Only a visitor's first name is shown. The email is stored but never sent back to the page.

Sign-in and passwords

Passwords are bcrypt hashes. A session cookie proves I'm signed in, and every write route checks it first.

MY IMPLEMENTATION

FILES AND SECRETS

Videos and images skip my server

1 The studio asks my server: may I upload?

2 The server signs the request with the secret key.

3 The browser sends the file straight to Cloudinary.

4 Cloudinary returns a URL. Only that URL is saved in the database.

Secrets stay out of Git

Database address, session secret, Cloudinary and Claude keys live in .env.

.env is in .gitignore, so it's never committed. The repo has .env.example with placeholders.

MY IMPLEMENTATION

HOW I BUILT IT

How I used AI

I wrote a spec mapping every brief requirement to a feature, gave it to Claude Code, and built in three parts, reviewing each before the next.

Docs I used

Neon's docs and its ready-made skills for Claude Code, Cloudinary's upload docs, Anthropic's API docs, and NOAA's public Kp data feed.

Problems I hit

Intuitive navigation: a 3D cube is new to visitors, and my first controls sat crammed against the frame. Fix: centred arrows, a key hint and the overview in the header.

Cube on top: the cube's stacked layers covered the page, so links like social media couldn't be clicked. Fix: only the visible face takes clicks.

RECAP

LEFT OUT ON PURPOSE

Sessions live in server memory	A server restart signs me out.
No login rate limiting	Someone could keep guessing passwords without being slowed down.
No comment moderation yet	Comments go live the moment they’re sent.

RECAP

FOUR THINGS TO REMEMBER

1. Data lives in Neon: three tables for users, posts and comments.
2. It gets there through fetch(), an Express route and an SQL query.
3. Placeholders, server checks, private emails and hashed passwords keep it safe.
4. Files go to Cloudinary; secrets stay in .env.

QUESTIONS?